

SQL Subqueries

18 · Subqueries — Complete Reference Guide

Table of Contents

1. Result Types
2. Subquery in FROM Clause
3. Subquery in SELECT
4. Subquery in JOIN Clause
5. Comparison Operators
6. IN Operator
7. ANY Operator
8. Correlated Subquery
9. EXISTS Operator

1. Subquery Result Types

→ *Scalar Query — returns a single value*

```
SELECT AVG(Sales) FROM Sales.Orders;
```

→ *Row Query — returns a single column of values*

```
SELECT CustomerID FROM Sales.Orders;
```

→ *Table Query — returns multiple columns and rows*

```
SELECT OrderID, OrderDate FROM Sales.Orders;
```

2. Subquery in FROM Clause

→ *Task 1: Products with price higher than the average price*

```
SELECT *  
FROM (
```

```

SELECT
    ProductID,
    Price,
    AVG(Price) OVER () AS AvgPrice
FROM Sales.Products
) AS t
WHERE Price > AvgPrice;

```

→ *Task 2: Rank customers based on their total sales*

```

SELECT
    *,
    RANK() OVER (ORDER BY TotalSales DESC) AS CustomerRank
FROM (
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
) AS t;

```

3. Subquery in SELECT

→ *Task 3: Show product details along with the total number of orders*

```

SELECT
    ProductID,
    Product,
    Price,
    (SELECT COUNT(*) FROM Sales.Orders) AS TotalOrders
FROM Sales.Products;

```

4. Subquery in JOIN Clause

→ *Task 4: Customer details with their total sales*

```

SELECT
    c.*,
    t.TotalSales
FROM Sales.Customers AS c
LEFT JOIN (
    SELECT
        CustomerID,
        SUM(Sales) AS TotalSales
    FROM Sales.Orders
    GROUP BY CustomerID
) AS t
ON c.CustomerID = t.CustomerID;

```

→ *Task 5: Customer details with their total number of orders*

```
SELECT
    c.*,
    o.TotalOrders
FROM Sales.Customers AS c
LEFT JOIN (
    SELECT
        CustomerID,
        COUNT(*) AS TotalOrders
    FROM Sales.Orders
    GROUP BY CustomerID
) AS o
ON c.CustomerID = o.CustomerID;
```

5. Subquery with Comparison Operators

→ *Task 6: Products with price higher than the average price*

```
SELECT
    ProductID,
    Price,
    (SELECT AVG(Price) FROM Sales.Products) AS AvgPrice
FROM Sales.Products
WHERE Price > (SELECT AVG(Price) FROM Sales.Products);
```

6. Subquery with IN Operator

→ *Task 7: Orders made by customers in Germany*

```
SELECT *
FROM Sales.Orders
WHERE CustomerID IN (
    SELECT CustomerID
    FROM Sales.Customers
    WHERE Country = 'Germany'
);
```

→ *Task 8: Orders made by customers NOT in Germany*

```
SELECT *
FROM Sales.Orders
WHERE CustomerID NOT IN (
    SELECT CustomerID
    FROM Sales.Customers
    WHERE Country = 'Germany'
);
```

7. Subquery with ANY Operator

→ *Task 9: Female employees earning more than ANY male employee*

```
SELECT EmployeeID, FirstName, Salary
FROM Sales.Employees
WHERE Gender = 'F'
      AND Salary > ANY (
      SELECT Salary
      FROM Sales.Employees
      WHERE Gender = 'M'
      );
```

8. Correlated Subquery

→ *Task 10: Total orders per customer using a correlated subquery*

```
SELECT
    *,
    (SELECT COUNT(*)
     FROM Sales.Orders o
     WHERE o.CustomerID = c.CustomerID) AS TotalSales
FROM Sales.Customers AS c;
```

9. Subquery with EXISTS Operator

→ *Task 11: Orders made by customers in Germany (using EXISTS)*

```
SELECT *
FROM Sales.Orders AS o
WHERE EXISTS (
    SELECT 1
    FROM Sales.Customers AS c
    WHERE Country = 'Germany'
          AND o.CustomerID = c.CustomerID
    );
```

→ *Task 12: Orders made by customers NOT in Germany (using NOT EXISTS)*

```
SELECT *
FROM Sales.Orders AS o
WHERE NOT EXISTS (
    SELECT 1
    FROM Sales.Customers AS c
    WHERE Country = 'Germany'
          AND o.CustomerID = c.CustomerID
    );
```

